Reporte

Nombre: Giovani Aldahir Patiño Rodriguez **Nombre**: Aldo Rene Romero Parga **Nombre**: Leonardo Alonso Muñiz Espinosa	**Matrícula**: 2118917 **Matrícula**: 2118748 **Matrícula**: 2115635
Nombre del curso: Componentes de Logica de Negocio	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: U3-FO3	**Actividad**: Diseño Arquitectural Del Sistema
Fecha: 13/10/2026	
Bibliografia **FIME UANL. (2026).** ***Caso 04: VetCare 24 (Unidad 2: Análisis de Requerimientos de Negocio)***	

01. Elegir Patrones de Lógica de Negocio

Tabla Comparativa de Patrones de Lógica de Negocio

Criterio	Transaction Script	Table Module	Domain Model (DDD - Rich Domain)
Complejidad Tecnológica	Baja. Lógica procedimental organizada mediante scripts o funciones por transacción.	Media. Organizada alrededor de las tablas de la base de datos relacional.	Alta. Orientada a objetos rica, encapsula comportamiento, estado e invariantes del negocio.
Manejo de Reglas Complejas	Dificultad severa para mantener reglas entrelazadas (e.g., inmutabilidad clínica, validación cruzada).	Duplicación de lógica entre diferentes módulos de consulta o procedimientos.	Ideal para reglas entrelazadas (RN-01, RN-04, RN-06) y dominios con alta lógica de estado.
Mantenibilidad a Largo Plazo	Baja. Se degrada rápidamente a código "espagueti" al incrementarse los requerimientos.	Media. Altamente acoplado al esquema de base de datos relacional.	Alta. Desacopla la lógica de negocio de la infraestructura y persistencia.
Aplicabilidad en VetCare 24	**Descartado.** No refleja la complejidad de la relación Tutor-Paciente ni garantiza inmutabilidad médica.	**Descartado.** Riesgo crítico de vulnerar reglas de seguridad clínica por dispersión de validaciones.	**SELECCIONADO.** Garantiza consistencia transaccional y cumplimiento normativo estricto.

Justificación de la Selección Arquitectónica (Domain Model / DDD)

Para el sistema **VetCare 24**, se ha seleccionado el patrón **Domain Model** (*Rich Domain Model*) enmarcado en la metodología **Domain-Driven Design (DDD)**. La complejidad operativa de un hospital veterinario 24/7 no reside únicamente en persistir datos en tablas de base de datos, sino en garantizar reglas clínicas y de seguridad críticas en tiempo real. Las razones técnicas y estratégicas que fundamentan esta decisión son:

1. **Encapsulamiento de Invariantes y Modelos Ricos frente a Modelos Anémicos:**
    - En enfoques procedimentales como *Transaction Script*, los datos son simples contenedores pasivos (DTOs) y la lógica de negocio se dispersa en servicios de aplicación o controladores. Esto incrementa exponencialmente el riesgo de omitir validaciones críticas.
    - Con DDD, las entidades y agregados (**Domain Model**) encapsulan tanto el estado como los métodos que garantizan la validez interna del sistema.
    - **RN-01 (Inmutabilidad Médica):** La entidad Consulta no permite la modificación directa de una NotaSOAP tras su firma. La inmutabilidad se autoprotege en el dominio exigiendo la instanciación formal de una NotaEnmienda vinculada.
    - **RN-04 (Bloqueo Estricto por Alergia):** Al agregar un medicamento a una RecetaMedica, la entidad evalúa internamente los alérgenos del Paciente. Si existe coincidencia, el mismo agregado interrumpe la ejecución arrojando un evento/excepción de dominio, imposibilitando que código externo salte esta validación.
2. **Gestión del Ciclo de Vida Complejo e Internamiento 24/7:**
    - La atención médica hospitalaria implica estados de transición continuos: admisión por urgencia, registro periódico de constantes cada 2–4 horas, y egreso coordinado.
    - **RN-06 (Validación Cruzada de Alta Hospitalaria):** El proceso de liberación de una jaula requiere la convergencia estricta de dos confirmaciones independientes (AltaMedicaFirmada por el veterinario y CuentaLiquidada por recepción). El agregado Hospitalizacion actúa como guardián del estado, asegurando que la jaula solo pase a estado "Libre" cuando ambas condiciones se cumplan.

3. **Desacoplamiento de Infraestructura y Testabilidad:**
    - DDD permite aislar las reglas operativas de los detalles técnicos como bases de datos, ORMs o marcos de interfaz de usuario. La lógica médica se puede probar mediante pruebas unitarias exhaustivas sin necesidad de conectar una base de datos o levantar un servidor web.
4. **Persistencia e Historial Inalterable (RN-05):**
    - En el evento de que un Paciente sea transferido a un nuevo TutorResponsable, la relación legal se actualiza dentro del agregado, pero la entidad conserva intacto el historial médico completo, asegurando continuidad asistencial y trazabilidad legal.

02. Mapa de Dominio (DDD) y Lenguaje Ubicuo

2.1 Lenguaje Ubicuo (Ubiquitous Language)

El Lenguaje Ubicuo establece un vocabulario técnico riguroso y compartido entre el equipo de desarrollo de software y los expertos del dominio veterinario (Directores Médicos, Veterinarios, Recepcionistas y Enfermería):

1. **Tutor / Propietario:** Persona física registrada con identificación oficial y datos de contacto, titular legal de la cuenta familiar y responsable financiero de uno o varios pacientes.
2. **Paciente (Mascota):** Animal individualizado que recibe atención médica. Posee un expediente clínico único e inalterable, opcionalmente vinculado a un microchip subcutáneo.
3. **Expediente Clínico Digital:** Conjunto consolidado, inmutable y cronológico de la historia médica, diagnósticos, recetas, alergias y bitácoras del paciente.
4. **Triaje Veterinario:** Protocolo de clasificación de urgencia por niveles (Rojo = Emergencia crítica, Amarillo = Urgencia, Verde = Rutina).
5. **Código Rojo / Admisión de Urgencia:** Flujo prioritario para pacientes con riesgo vital inminente que omite trámites administrativos de recepción para ingresar de inmediato a sala de choque.
6. **Nota SOAP:** Registro clínico estandarizado (Subjetivo, Objetivo, Análisis, Plan) firmado por el médico tratante, quedando en estado congelado.
7. **Nota de Enmienda (Fe de Erratas):** Adenda normativa para corregir o complementar una nota clínica previamente firmada, conservando intacto el registro original.

8. **Calendario Profiláctico:** Cronograma dinámico de intervenciones preventivas obligatorias (vacunas y desparasitaciones).
9. **Refuerzo (Booster):** Dosis subsecuente de un biológico requerida tras un intervalo específico (21, 28 o 365 días).
10. **Lote de Biológico:** Código del fabricante impreso en el biológico, obligatorio antes de registrar la aplicación para asegurar trazabilidad.
11. **Alerta de Alergia / Bloqueo Severo:** Mecanismo restrictivo del sistema que detiene la prescripción si el principio activo coincide con una alergia registrada.
12. **Hospitalización (24/7):** Régimen de internamiento continuo en instalaciones clínicas para fluidoterapia, terapia intensiva o monitoreo postquirúrgico.
13. **Bitácora Hospitalaria Digital:** Registro continuo de constantes vitales (temperatura, frecuencia cardíaca, frecuencia respiratoria) y medicamentos capturado cada 2 a 4 horas.
14. **Alta Médica y Validación Cruzada:** Dictamen formal de egreso emitido por el veterinario que requiere confirmación de pago en recepción para autorizar la salida física y liberar la jaula.

2.2 Descomposición en Agregados (Aggregates) y Contextos Delimitados

1. Agregado: PacienteAggregate

- **Raíz del Agregado (Aggregate Root):** Paciente
- **Entidades Internas:** HistorialAlergias, TutorResponsable.
- **Value Objects (VO):** PacienteId, MicrochipCode, Peso, Alergeno.
- **Invariantes y Reglas de Negocio:**
    - Prohibición de duplicidad de MicrochipCode activo.
    - Cualquier actualización de peso o alérgeno emite una actualización al perfil general.
    - La transferencia de tutor preserva el historial médico completo del paciente intacto (RN-05).
- **Domain Events:** PacienteRegistradoEvent, AlergiaDetectadaEvent, TutorReasignadoEvent.

2. Agregado: ConsultaAggregate

- **Raíz del Agregado (Aggregate Root):** Consulta

- **Entidades Internas:** NotaSOAP, NotaEnmienda, RecetaMedica.
- **Value Objects (VO):** ConsultaId, CedulaProfesional, Diagnostico, MedicamentoDosificado.
- **Invariantes y Reglas de Negocio:**
    - **RN-01 (Inmutabilidad Médica):** Al firmar la NotaSOAP, el registro pasa a estado "Congelado".
    - **Nota de Enmienda:** Toda corrección requiere instanciar una NotaEnmienda con cédula, fecha, hora y motivo.
    - **RN-04 (Bloqueo por Alergia):** Interrupción automática e inexpugnable ante la prescripción de sustancias contraindicadas.
- **Domain Events:** NotaSOAPFirmadaEvent, NotaEnmiendaAnexadaEvent, BloqueoAlergiaDisparadoEvent.

3. Agregado: HospitalizacionAggregate

- **Raíz del Agregado (Aggregate Root):** Hospitalizacion
- **Entidades Internas:** BitacoraSignos, PrecuentaConsumos.
- **Value Objects (VO):** JaulaId, ConstantesVitales, EstadoAlta.
- **Invariantes y Reglas de Negocio:**
    - Registro de constantes obligatorio en ventanas máximas de 4 horas (2 horas en estado crítico).
    - **RN-06 (Validación Cruzada):** Liberación de jaula condicionada a AltaMedicaFirmada = True y CuentaLiquidada = True.
- **Domain Events:** PacienteHospitalizadoEvent, SignosVitalesFueraDeRangoEvent, AltaMedicaAutorizadaEvent, CuentaLiquidadaEvent, JaulaLiberadaEvent.

4. Agregado: ProfilaxisAggregate

- **Raíz del Agregado (Aggregate Root):** RegistroVacuna
- **Entidades Internas:** ProgramacionRefuerzo.
- **Value Objects (VO):** LoteBiologico, MarcaVacuna, IntervaloRefuerzo.
- **Invariantes y Reglas de Negocio:**
    - **RN-03 (Trazabilidad Obligatoria):** Imposible asentar una vacuna sin capturar LoteBiologico y MarcaVacuna.

  - Cálculo automatizado de fecha de próximo refuerzo al asentar la aplicación.

- **Domain Events:** VacunaAplicadaEvent, RefuerzoProfilacticoCalculadoEvent, NotificacionRecordatorioDespachadaEvent.

03. Capa de Servicios de Aplicación (Service Layer)

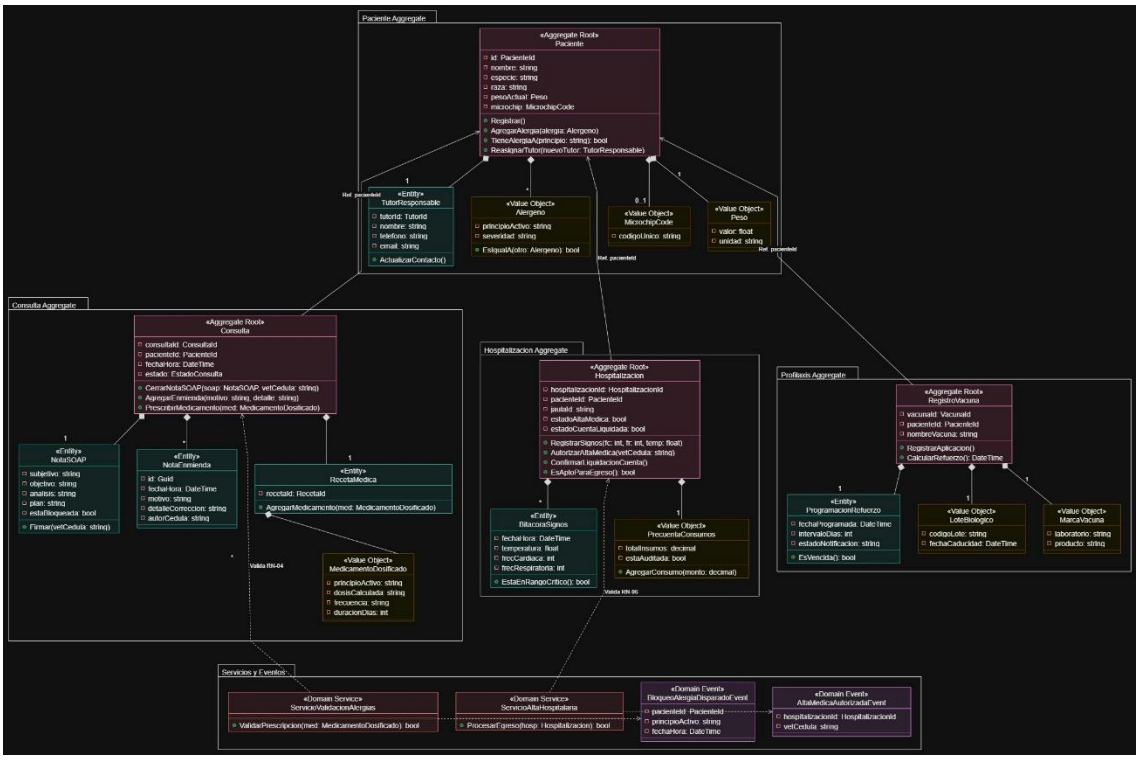
La capa de servicios actúa como orquestadora de flujos sin ejecutar lógica de negocio propia, coordinando entidades, repositorios y eventos:

- **Módulo 1: Registro de Pacientes e Identidad Única**
  - US01: RegistrarMascota(DatosTutor, DatosMascota) -> Instancia Paciente y asigna TutorResponsable.
  - US12: BuscarPacientesPredictivo(CriterioBusqueda) -> Retorna consulta optimizada PacienteDTO.
- **Módulo 2: Agenda Inteligente y Triaje**
  - US03: ActivarAdmisionUrgencia(DescripcionTrauma) -> Instancia pre-expediente temporal e imparte señal de Código Rojo (RN-02).
  - US04: CapturarTriajeInicial(PacienteId, PesoRegistrado, AlergiaReportada) -> Delega actualización de peso y alérgenos al dominio.
- **Módulo 3: Expediente Clínico y Prescripción**
  - US05: FirmarYCerrarNotaSOAP(ConsultaId, ContenidoSOAP, CedulaVeterinario) -> Invoca la congelación inmutable de la nota (RN-01).
  - US06: PrescribirMedicamento(ConsultaId, PacienteId, PrincipioActivo, Dosis) -> Ejecuta la validación de alergias del paciente (RN-04).
  - US13: AnexarNotaEnmienda(ConsultaId, Motivo, DetalleCorreccion, CedulaVeterinario) -> Conecta la adenda de aclaración sin alterar la nota original.
- **Módulo 4: Control Profiláctico (Vacunación)**
  - US07: RegistrarAplicacionVacuna(PacienteId, NombreVacuna, NumeroLote, Marca) -> Valida trazabilidad e instruye cálculo de refuerzo (RN-03).

- US08: ProcesarRecordatoriosPendientes() -> Coordina el envío de avisos (WhatsApp/Correo) 48 horas antes del vencimiento.

- **Módulo 5: Hospitalización y Operaciones**
  - US09: IngresarAHospitalizacion(PacienteId, JaulaId, PlanFluidoterapia) -> Cambia estado de jaula a ocupada e inicia régimen.
  - US10: RegistrarConstantesVitales(HospitalizacionId, SignosVitales) -> Añade lectura a la bitácora y evalúa rangos de alerta.
  - US11: ProcesarAltaHospitalaria(HospitalizacionId, AccionOrigen) -> Valida la convergencia cruzada (Médica + Financiera) para liberar jaula (RN-06).

04. Diagramas y Registros de Decisión de Arquitectura (ADR)



4.1. **Justificación del Mapa de Dominio (Diagrama de Clases DDD)**

El diseño del diagrama de clases sigue estrictamente los principios de **desacoplamiento, consistencia transaccional y claridad visual**:

A. Descomposición en 4 Agregados Delimitados

El sistema se dividió en cuatro Agregados independientes (PacienteAggregate, ConsultaAggregate, HospitalizacionAggregate y ProfilaxisAggregate), cada uno con un límite explícito de consistencia (*Boundary*):

- **PacienteAggregate:** Concentra la identidad, datos fisiológicos cambiantes (Peso), identificación subcutánea (MicrochipCode) y el catálogo inmutable de alergias (Alergeno).
- **ConsultaAggregate:** Encapsula la atención ambulatoria, la nota clínica SOAP, sus adendas de enmienda y la receta médica.
- **HospitalizacionAggregate:** Governa el internamiento 24/7, las lecturas periódicas de constantes vitales cada 2-4 horas (BitacoraSignos) y la auditoría de consumos (PrecuentaConsumos).
- **ProfilaxisAggregate:** Administra la aplicación de vacunas, forzando la captura de trazabilidad (LoteBiologico y MarcaVacuna) y calculando los refuerzos profilácticos futuros.

B. Referenciación por Identificador (pacienteId) y Desacoplamiento

- **Elección de Diseño:** Los Agregados independientes **no se contienen directamente entre sí mediante relaciones de composición o asociación dura** en el código. En su lugar, Consulta, Hospitalizacion y RegistroVacuna hacen referencia al Agregado principal únicamente mediante el atributo pacienteId.
- **Justificación Técnica:**
    - **Transacciones Atómicas Independientes:** Modificar una consulta médica o registrar una constante en la bitácora de hospitalización no requiere cargar en memoria ni bloquear la tabla completa del paciente, reduciendo contenciones en la base de datos.
    - **Claridad en la Diagramación:** Elimina las líneas cruzadas y enredadas que atravesaban el diagrama, garantizando una disposición ortogonal y limpia por niveles.

C. Uso Estratégico de Value Objects (Objetos de Valor)

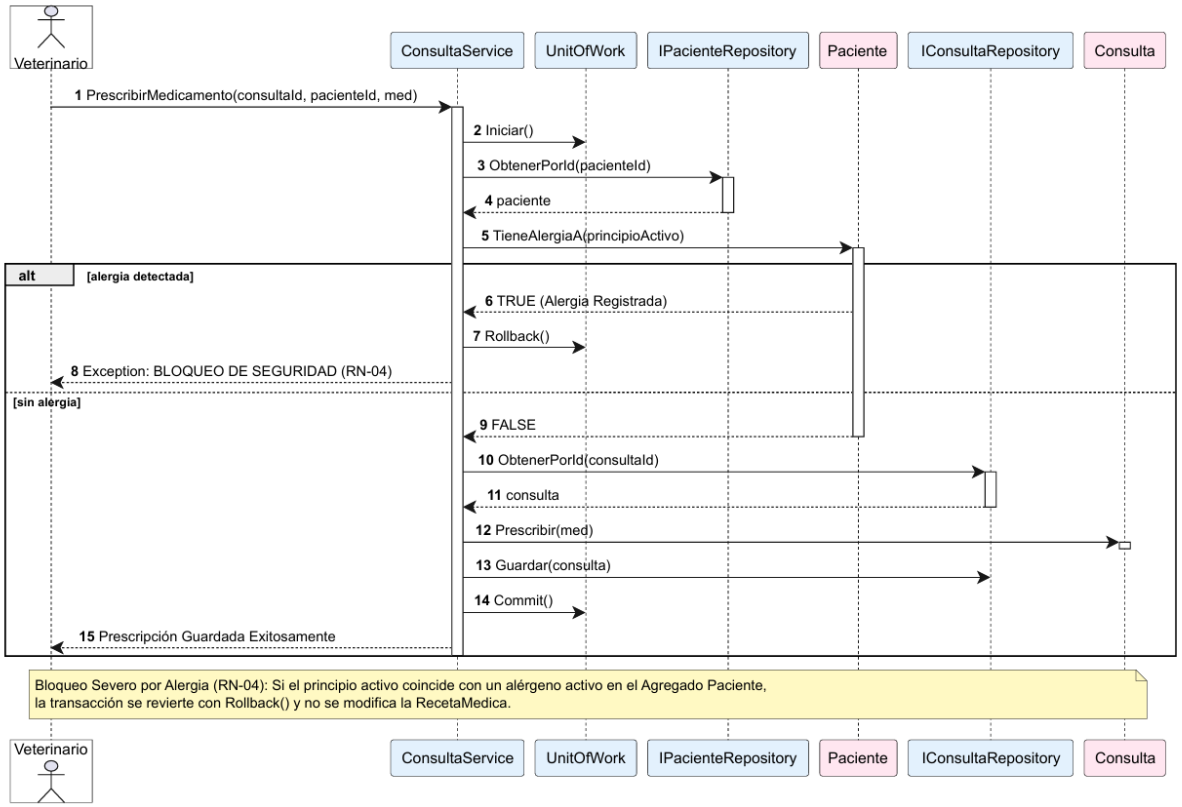
Elementos como Peso, MicrochipCode, Alergeno, MedicamentoDosificado, PrecuentaConsumos, LoteBiologico y MarcaVacuna se diseñaron como **Value Objects inmutables**:

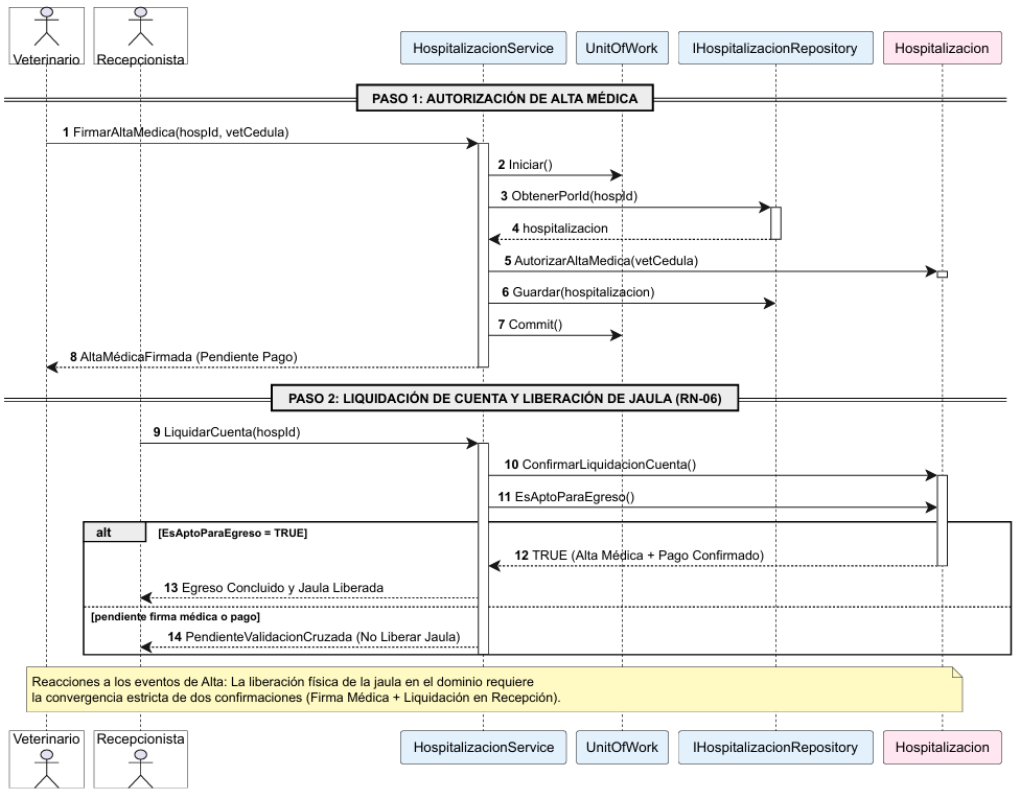
- **Justificación:** Un valor de peso o una dosis no tienen identidad propia en el tiempo; si cambian, se reemplazan por una nueva instancia completa. Esto previene mutaciones colaterales imprevistas en la memoria de la aplicación.

D. Separación de Servicios y Eventos de Dominio (Services y Eventos)

Las operaciones que involucran la interacción de múltiples agregados o la notificación asíncrona hacia otros módulos se abstrajeron en la capa de servicios de dominio:

- **ServicioValidacionAlergias:** Coordina la evaluación de seguridad clínica entre Paciente y Consulta, publicando el evento BloqueoAlergiaDisparadoEvent.
- **ServicioAltaHospitalaria:** Audita el cumplimiento de la RN-06 y publica AltaMedicaAutorizadaEvent.





4.2. Justificación de los Diagramas de Secuencia (DDS-01 y DDS-02)

Los diagramas de secuencia ilustran la orquestación síncrona y transaccional entre la Capa de Servicios de Aplicación, los Repositorios y el Dominio.

A. Adopción del Patrón UnitOfWork y Repository

En ambos diagramas se introdujeron explícitamente las abstracciones UnitOfWork y Repository (IPacienteRepository, IConsultaRepository, IHospitalizacionRepository):

- **Justificación:** La Capa de Servicios (ConsultaService / HospitalizacionService) no ejecuta consultas SQL ni gestiona directamente conexiones a la base de datos. Delegar el inicio (Iniciar()), la confirmación (Commit()) y la cancelación (Rollback()) a un UnitOfWork garantiza la **atomicidad transaccional** (si algo falla en el dominio, nada se guarda en la BD).

B. Secuencia RN-04: Prescripción Segura y Bloqueo por Alergias

1. **Evaluación Previa al Cambio de Estado:** La capa de servicio invoca TieneAlergiaA() sobre la entidad Paciente **antes** de intentar modificar la RecetaMedica dentro de la Consulta.

2. **Manejo de Excepciones y Rollback Inmediato:** Si la respuesta es TRUE, el sistema ingresa al bloque alternativo (alt [alergia detectada]), ejecuta inmediatamente UOW.Rollback() y arroja una excepción de dominio (BLOQUEO DE SEGURIDAD RN-04).
3. **Justificación:** Se garantiza que ante cualquier intento de recetar un fármaco contraindicado, la transacción se cancele a nivel de memoria antes de tocar la base de datos, protegiendo la salud del paciente.

C. Secuencia RN-06: Alta Hospitalaria y Validación Cruzada

1. **Arquitectura en Dos Fases:**
    - **PASO 1 (Médico):** El veterinario ejecuta FirmarAltaMedica(), lo que cambia la bandera estadoAltaMedica = true en el Agregado Hospitalizacion, dejando el expediente en estado "Pendiente de Pago".
    - **PASO 2 (Recepción):** El recepcionista ejecuta LiquidarCuenta(), cambiando estadoCuentaLiquidada = true.
2. **Evaluación de Concurrencia de Egreso:** El servicio consulta el método de dominio EsAptoParaEgreso().
3. **Bloque Alternativo de Seguridad:**
    - Si ambas confirmaciones están en verdadero (TRUE), se concluye el egreso y se emite la señal de **Jaula Liberada**.
    - Si falta la firma médica o la liquidación, el dominio devuelve el estado **PendienteValidacionCruzada**, impidiendo la liberación del espacio físico.
4. **Justificación:** Resuelve de forma definitiva los conflictos operacionales y pérdidas financieras entre el área clínica y la recepción, asegurando que ningún paciente sea egresado sin la debida alta médica y el saldo liquidado.

REGISTRO DE DECISIÓN DE ARQUITECTURA (ADR-001)

ADR-001: Selección del Patrón de Lógica de Negocio y Arquitectura de Dominio para VetCare 24

- **Estatus:** Aprobado
- **Fecha:** 2026-10-13
- **Autores:** Equipo 4 — Componentes de Lógica de Negocio (Giovani Patiño, Aldo Romero, Leonardo Muñiz)
- **Institución:** Facultad de Ingeniería Mecánica y Eléctrica (FIME - UANL)
- **Proyecto:** Sistema de Gestión Clínica y Hospitalaria 24/7 (VetCare 24)

1. Contexto y Declaración del Problema

El sistema **VetCare 24** está diseñado para gestionar operativamente una clínica y centro hospitalario veterinario con atención médica ininterrumpida (24/7). El modelo de negocio abarca desde atención primaria de urgencias y agenda diferenciada, hasta el control de expedientes clínicos inmutables, la administración de biológicos y la hospitalización crítica con monitoreo continuo.

Durante el análisis del modelo operativo (AS-IS) y el levantamiento de requerimientos con el Director Médico y el Jefe de Operaciones, se identificaron las siguientes **Reglas de Negocio Inexpugnables (RN-01 a RN-06)**:

1. **RN-01 (Inmutabilidad Médica Estricta):** Una nota de evolución clínica (NotaSOAP) firmada por el veterinario se congela de manera inalterable. No se admiten ediciones directas ni borrados; cualquier corrección debe instanciar una NotaEnmienda (adenda) vinculada con registro explícito de cédula, fecha, hora y motivo.
2. **RN-02 (Prioridad de Triage Rojo / Código Rojo):** Los pacientes en riesgo vital inminente saltan la fila de espera y los trámites administrativos de cobro para ingresar de inmediato a estabilización en quirófano o sala de choque, generando un pre-expediente temporal de intervención.
3. **RN-03 (Trazabilidad Obligatoria de Biológicos):** Es imposible asentar la aplicación de una vacuna sin capturar formalmente el código de LoteBiologico y la MarcaVacuna para fines de farmacovigilancia.
4. **RN-04 (Bloqueo Severo por Alergias Activas):** El sistema debe interrumpir automáticamente la confirmación de una receta médica si el principio activo prescrito coincide con un alérgeno registrado en el perfil del paciente, impidiendo accidentes de mala praxis o choques anafilácticos.

5. **RN-05 (Propiedad del Historial Clínico):** El expediente le pertenece a la mascota. Ante una transferencia de titularidad (cambio de TutorResponsable), el historial de diagnósticos, cirugías y alergias se conserva intacto.
6. **RN-06 (Validación Cruzada de Alta Hospitalaria):** La liberación física de una jaula hospitalaria exige la convergencia de dos firmas independientes: el dictamen de AltaMedicaFirmada por el médico tratante y la confirmación de CuentaLiquidada emitida por recepción tras auditar la precuenta de consumos.

El problema arquitectónico que resolver es: ¿Qué patrón de diseño debe gobernar la capa de lógica de negocio para garantizar el cumplimiento transaccional de estas reglas, evitar fugas de lógica de seguridad clínica hacia la base de datos o la interfaz de usuario, y mantener el sistema escalable y altamente testable a largo plazo?

2. Impulsores Arquitectónicos (Architectural Drivers)

El diseño del software está fuertemente restringido por atributos de calidad y requerimientos funcionales críticos:

- **Seguridad y Auditoría Clínica (Safety & Auditability):** Imposibilidad de alterar notas pasadas o recetar fármacos contraindicados.
- **Consistencia Transaccional (Transactional Consistency):** La validación cruzada de egreso hospitalario (RN-06) y la trazabilidad de lotes (RN-03) deben ejecutarse atómicamente.
- **Rendimiento en Urgencias (RNF-01):** Apertura y despliegue del expediente de triage en menos de 3 segundos.
- **Desacoplamiento Tecnológico:** La lógica médica no debe depender de bibliotecas de interfaz gráfica ni del ORM/motor de base de datos relacional.
- **Mantenibilidad y Testabilidad Unitaria:** Capacidad de probar las reglas de vacunación, dosis y alergias de forma aislada mediante pruebas unitarias puras.

3. Opciones Consideradas

Se evaluaron tres patrones fundamentales de arquitectura para la capa de lógica de negocio (según el catálogo de Martin Fowler):

Opción 1: Transaction Script (Patrón Procedimental)

Organizar la lógica en funciones o procedimientos lineales por cada transacción del sistema (e.g., PrescribirMedicamentoScript(), AltaHospitalariaScript()).

- **Pros:** Simplicidad inicial y baja curva de aprendizaje.
- **Contras:** La lógica de validación de alergias (RN-04) e inmutabilidad (RN-01) se duplica en múltiples scripts. Con el tiempo degenera en código "espagueti", haciendo frágil la aplicación y propensa a saltarse reglas de seguridad médica.

Opción 2: Table Module (Orientado a Tablas)

Estructurar la lógica de negocio alrededor de las tablas relacionales de la base de datos (e.g., una clase de negocio por cada tabla de la BD como PacientesTableModule, ConsultasTableModule).

- **Pros:** Encaja naturally con bases de datos relacionales y estructuras DataSet/tabulares.
- **Contras:** Fuerte acoplamiento al esquema físico de la base de datos relacional. Dificultad para modelar entidades complejas compuestas y reglas entrelazadas en tiempo real como la validación cruzada en hospitalización (RN-06).

Opción 3: Domain Model (Rich Domain Model / Domain-Driven Design - DDD)

Modelar un red de objetos de dominio donde las entidades y agregados encapsulan tanto el estado como el comportamiento y las invariantes del negocio, guiados por un **Lenguaje Ubicuo**.

- **Pros:** Las reglas de negocio (RN-01 a RN-06) residen dentro de los propios Agregados (PacienteAggregate, ConsultaAggregate, HospitalizacionAggregate, ProfilaxisAggregate), autoprotegiéndose contra estados inválidos. Altísima testabilidad pura sin base de datos.
- **Contras:** Mayor complejidad conceptual inicial, necesidad de patrones de mapeo de persistencia (Repositories) y curva de aprendizaje para el equipo.

4. Matriz Comparativa de Evaluación

Criterio de Evaluación	Transaction Script	Table Module	Domain Model (DDD - Rich Domain)
Complejidad Tecnológica	Baja	Media	Alta
Encapsulamiento de Invariantes (RN-01 a RN-06)	Vulnerable (Riesgo de eludir validaciones)	Moderado (Acoplado a tablas relacionales)	**Robusto e Inexpugnable** (Protegido en la Raíz del Agregado)
Mantenibilidad a Largo Plazo	Baja (Degrada a código espagueti)	Media	**Alta** (Desacoplado de infraestructura)
Testabilidad Unitaria	Dificultad para aislar la base de datos	Requiere BD relacional para probar	**Excelente** (Pruebas unitarias sobre objetos puros)
Representación del Lenguaje Ubicuo	Nula (Estructurado por consultas)	Parcial (Estructurado por tablas)	**Total** (Mapeo directo del dominio médico)
Garantía de Inmutabilidad (RN-01)	Difícil de auditar	Riego de UPDATE directo en tabla	**Garantizada** (Sin setters públicos, adendas explícitas)

5. Decisión Seleccionada

Se selecciona la **Opción 3: Domain Model bajo la metodología Domain-Driven Design (DDD)**.

Componentes de la Solución Adoptada:

1. **Descomposición en 4 Agregados Delimitados (Aggregates):**
    - **PacienteAggregate:** Protege la identidad única, el peso actualizado, el microchip y la colección inalterable de alérgenos.

- **ConsultaAggregate:** Garantiza la inmutabilidad de la NotaSOAP, la emisión de NotaEnmienda auditada y la prescripción farmacológica.
- **HospitalizacionAggregate:** Modela el ciclo de vida de la estancia 24/7, la bitácora de signos vitales cada 2-4 horas y la confirmación cruzada de egreso.
- **ProfilaxisAggregate:** Asegura la captura obligatoria de lote/marca (RN-03) y calcula los refuerzos futuros.

2. **Capa de Servicios de Aplicación delgada (Service Layer):**
   - Coordina los flujos de uso (US01 a US15), gestiona transacciones mediante UnitOfWork y delega la ejecución de reglas de negocio al Dominio sin ejecutar validaciones directas.
3. **Domain Services y Eventos de Dominio:**
   - Uso de ServicioValidacionAlergias y ServicioAltaHospitalaria para coordinar flujos complejos inter-agregado y publicación de eventos como BloqueoAlergiaDisparadoEvent y AltaMedicaAutorizadaEvent.

6. Consecuencias del Diseño

Consecuencias Positivas:

- **Autoprotección del Dominio:** Imposible persistir una receta con sustancias alérgicas o dar de alta a un paciente sin firma médica y saldo en cero, ya que los agregados arrojan excepciones de dominio antes de tocar la base de datos.
- **Auditoría e Inmutabilidad Médica:** Cumplimiento normativo de la norma legal médica al forzar el uso de notas de enmienda vinculadas.
- **Pruebas Rápidas y Confiables:** Cobertura de pruebas unitarias sobre las reglas clínicas en milisegundos, al no depender de frameworks o infraestructura de persistencia.
- **Alineación con Stakeholders:** Trazabilidad directa entre el código fuente, los diagramas UML y las definiciones vertidas por el cuerpo médico durante las entrevistas.

Consecuencias Negativas y Estrategias de Mitigación:

- **Riesgo 1: Complejidad en el Mapeo Objeto-Relacional (ORM).**

- *Mitigación:* Se implementa el patrón **Repository** aislando las clases de mapeo físico de las entidades del dominio rico.

- **Riesgo 2: Fugas de Lógica de Negocio hacia los Servicios de Aplicación.**
  - *Mitigación:* Revisiones de código strictly para asegurar que la Service Layer solo orqueste (cargar agregado → invocar método del agregado → guardar agregado).

- **Riesgo 3: Curva de aprendizaje técnica para nuevos desarrolladores.**
  - *Mitigación:* Documentación estandarizada del Lenguaje Ubicuo y diagramas de clases y secuencia integrados en la solución.

7. Lineamientos de Gobernanza y Cumplimiento

Esta decisión arquitectónica se valida y monitorea bajo los siguientes criterios de ingeniería:

1. **Pruebas Unitarias del Dominio:** Mantenimiento de una cobertura mínima del 90% en métodos de agregados que evalúen reglas de negocio (TieneAlergiaA(), EsAptoParaEgreso(), Firmar()).
2. **Revisión de Código (Code Reviews):** Rechazo automático de PRs que incluyan lógica condicional de negocio dentro de controladores API o clases de la Service Layer.
3. **Estructura de Archivos en Repositorio GitHub:** Separación limpia de capas bajo arquitectura limpia (Domain, Application, Infrastructure, UI).

4.3. Justificación del ADR

1. Adopción de Domain-Driven Design (DDD) y Rich Domain Model

Para el sistema **VetCare 24**, se seleccionó el patrón **Domain Model** (*Rich Domain Model*) bajo la metodología **Domain-Driven Design (DDD)** frente a alternativas como *Transaction Script* o *Table Module*.

- **Eliminación del Modelo Anémico:** En un sistema de salud hospitalario 24/7, las validaciones operativas y médicas no pueden depender de scripts externos o controladores de API que olviden aplicar condicionales. El dominio rico concentra las reglas e invariantes dentro de sus propios objetos, impidiendo la persistencia de datos corruptos o inválidos.

- **Autoprotección de Reglas Inexpugnables:**

- **RN-01 (Inmutabilidad Médica):** La entidad Consulta deshabilita la edición directa de una NotaSOAP congelada, forzando la creación de adendas explícitas (NotaEnmienda).
- **RN-04 (Bloqueo Severo por Alergias):** La evaluación de alérgenos se ejecuta dentro del Agregado Paciente antes de modificar la receta médica, interrumpiendo la transacción directamente desde el dominio.
- **RN-06 (Validación Cruzada):** El Agregado Hospitalizacion controla las banderas de egreso, impidiendo la liberación física de la jaula si no coinciden la firma médica y la liquidación financiera.

2. Fundamentación del Mapa de Dominio (Diagrama de Clases DDD)

- **Descomposición en 4 Agregados Delimitados:**
  - **PacienteAggregate:** Gestiona la identidad única del animal, historial de alérgenos, microchip subcutáneo y datos del tutor legal.
  - **ConsultaAggregate:** Garantiza la inmutabilidad de notas SOAP, adendas de enmienda y recetas farmacológicas.
  - **HospitalizacionAggregate:** Modela el internamiento continuo 24/7, lecturas de constantes vitales en ventanas de 2 a 4 horas y auditoría de consumos.
  - **ProfilaxisAggregate:** Controla la aplicación de biológicos forzando la trazabilidad de lote y marca, y calcula la fecha de refuerzo futuro.
- **Desacoplamiento por Identificador (pacienteId):** Los Agregados independientes no se contienen entre sí mediante asociaciones duras en el código. Consulta, Hospitalizacion y RegistroVacuna hacen referencia al Agregado principal únicamente mediante el atributo pacienteId. Esto elimina bloqueos en la base de datos durante operaciones concurrentes y permite una diagramación limpia, ortogonal y sin cruces de líneas.
- **Uso de Value Objects Inmutables:** Elementos como Peso, MicrochipCode, Alergeno, MedicamentoDosificado, PrecuentaConsumos, LoteBiologico y MarcaVacuna se representan como Objetos de Valor inmutables. Si un dato cambia, la instancia completa se reemplaza, evitando modificaciones indeseadas en memoria.
- **Aislamiento de Servicios y Eventos de Dominio:** Operaciones complejas entre agregados se coordinan mediante servicios desacoplados (ServicioValidacionAlergias y ServicioAltaHospitalaria),

publicando eventos de dominio (BloqueoAlergiaDisparadoEvent, AltaMedicaAutorizadaEvent) para consumo asíncrono.

3. Fundamentación de los Diagramas de Secuencia (DDS-01 y DDS-02)

- **Uso de UnitOfWork y Repository:** Los servicios de aplicación (ConsultaService / HospitalizacionService) no gestionan conexiones ni ejecutan SQL directamente. La transacción completa se envuelve en un UnitOfWork que coordina los métodos Iniciar(), Commit() y Rollback(), garantizando atomicidad transaccional.
- **Secuencia RN-04 (Bloqueo por Alergia):** La verificación de contraindicaciones farmacológicas se invoca sobre el Agregado Paciente antes de modificar la RecetaMedica. En caso de detección positiva, el sistema ejecuta un Rollback() inmediato y emite una excepción de dominio, deteniendo el flujo antes de tocar la base de datos.
- **Secuencia RN-06 (Alta Hospitalaria):** El proceso se divide en dos fases independientes (Firma del Médico y Pago en Recepción). La liberación de la jaula hospitalaria solo ocurre cuando el método EsAptoParaEgreso() evalúa ambas banderas en TRUE. Si falta alguna confirmación, el sistema devuelve PendienteValidacionCruzada, reteniendo el espacio físico.

05. Bitácora de Uso de Inteligencia Artificial

Se presenta la bitácora técnica del diseño asistido por IA

#	Fase / Módulo	System Prompt / Entradas de Ingeniería Generadas por el Humano	Entregable / Resultado Generado por IA	Validación, Ajustes Técnicos y Criterios de Aceptación
01	**Modelado de Procesos (BPMN)**	"Actúa como Arquitecto de Software Senior especializado en BPMN 2.0. Genera el código XML nativo para Draw.io del diagrama de flujo del caso VetCare 24. Modela 3 carriles (Lanes): Recepción, Área Clínica y Hospitalización. Incluye compuertas de decisión exclusivas para Código Rojo (RN-02) y la validación cruzada de egreso hospitalario (RN-06). Exporta nodos estructurados con IDs unívocos."	Código XML nativo compatible con Draw.io conteniendo los 3 carriles, compuertas lógicas y flujo de adendas.	Importación directa en Draw.io verificada. Ajuste de geometría en conectores de compuertas paralelos y validación de nombres de actividades.
02	**Patrones de Lógica de Negocio**	"Como experto en patrones de arquitectura de Martin Fowler y Domain-Driven Design (DDD), elabora una tabla comparativa rigurosa entre Transaction Script, Table Module y Domain Model. Evalúa complejidad, mantenibilidad y manejo de reglas complejas. Justifica de manera exhaustiva por qué Rich Domain Model en DDD es	Tabla comparativa detallada y marco de justificación técnica enfocado en consistencia transaccional.	Confirmación del modelo rico frente al modelo anémico. Ampliación de la justificación para incluir escenarios de alta concurrencia y auditoría.

		la elección óptima para VetCare 24 basándote en RN-01 (Inmutabilidad), RN-04 (Bloqueo de Alergias) y RN-06 (Alta Cruzada)."		
03	**Lenguaje Ubicuo y Agregados (DDD)**	"Asume el rol de Domain Expert en Sistemas Veterinarios. Define formalmente el Lenguaje Ubicuo de VetCare 24 con 14 términos clave. Posteriormente, diseña la descomposición de 4 Agregados: PacienteAggregate, ConsultaAggregate, HospitalizacionAggregate y ProfilaxisAggregate. Para cada uno, especifica la Raíz del Agregado, Entidades, Value Objects, Invariantes explícitas (RN-01 a RN-06) y Domain Events producidos."	Glosario de Lenguaje Ubicuo y especificación técnica completa de los 4 agregados con sus invariantes y eventos.	Verificación estricta de que el PacienteId y MicrochipCode actúen como Value Objects y que las adendas no alteren el estado inmutable de las notas.
04	**Capa de Servicios de Aplicación**	"Diseña la Capa de Servicios de Aplicación (Service Layer) para VetCare 24 estructurada en 5 módulos (US01 a US13). Describe el método y flujo paso a paso de orquestación pura. RESTRICCIÓN ESTRICTA: La capa de servicio no debe ejecutar lógica de negocio ni evaluar reglas directamente; debe delegar todas las	Documentación técnica de la Service Layer detallando el flujo de orquestación pura por caso de uso.	Revisión de código/diseño para eliminar fugas de lógica de negocio en los servicios de aplicación, asegurando delegación total a las entidades.

		validaciones e invariantes a los agregados del dominio."		
05	**Diagramación UML y ADR**	"Genera el código XML para Draw.io del Diagrama de Clases de Dominio DDD y los Diagramas de Secuencia para las reglas críticas RN-04 (Bloqueo por Alergia) y RN-06 (Validación Cruzada de Alta). Adicionalmente, redacta el documento Architectural Decision Record (ADR-001) en formato Markdown justificando la adopción de DDD."	Tres bloques de código XML para diagramas UML y plantilla completa del documento ADR-001.	Importación exitosa en Draw.io. Ajuste de taxonomía de mensajes en secuencias y verificación de formato estándar Nygard en ADR-001.
06	**Documentación en GitHub**	"Genera una propuesta profesional de README.md para el repositorio GitHub de VetCare 24 en Markdown. Incluye badges de estado, tabla de contenidos, problema AS-IS vs solución TO-BE, estructura de carpetas de arquitectura hexagonal y la información institucional completa de la FIME UANL junto con los integrantes del Equipo 4."	Plantilla Markdown estructurada para el repositorio conteniendo la metadata institucional y la descripción del proyecto.	Validación de renderizado Markdown, actualización del enlace público del repositorio y comprobación de datos de matrícula del equipo.

06. Conclusión General del Diseño Arquitectónico

El diseño arquitectónico desarrollado para el sistema **VetCare 24** demuestra que la adopción de **Domain-Driven Design (DDD)** y el patrón **Rich Domain Model** es la solución óptima para entornos clínicos veterinarios de alta complejidad operativa. A través de esta arquitectura se logran los siguientes beneficios clave:

1. **Garantía Incondicional de Seguridad del Paciente:** La encapsulamiento de reglas críticas dentro de los agregados del dominio asegura que el sistema reaccione de manera automatizada e inexpugnable ante situaciones de riesgo vital, como el bloqueo inmediato por contraindicaciones de alergias (**RN-04**) y la trazabilidad obligatoria de biológicos (**RN-03**).
2. **Cumplimiento Normativo y Legal Médica:** La inmutabilidad de los expedientes clínicos (**RN-01**) mediante notas SOAP congeladas y el mecanismo de notas de enmienda garantiza la integridad legal de la información, auditabilidad completa y protección del historial médico ante eventualidades administrativas o cambios de propietario (**RN-05**).
3. **Eficiencia Operativa e Integración Transversal:** La orquestación en la capa de servicios y la convergencia de validaciones en hospitalización (**RN-06**) resuelven los fricciones administrativas tradicionales entre el cuerpo médico y el área de recepción, garantizando altas hospitalarias seguras y cobros auditados en tiempo real.

En conclusión, la arquitectura propuesta no solo resuelve las necesidades funcionales del hospital **VetCare 24**, sino que provee un software escalable, mantenible, desacoplado de la infraestructura subyacente y preparado para crecer ante futuros requerimientos de la institución.

07.Link Repositorio Github

Rakaseinon/U3-FO3_Dise-o-Arquitectural-Del-Sistema_EQUIPO4_VETCARE: Diseño arquitectónico y modelo de dominio (DDD) para VetCare 24 — Sistema de gestión clínica veterinaria 24/7 con arquitectura orientada a la lógica de negocio.